

Develop, Debug and Deploy NX-SDK Python Application in Nexus

Updated: June 8, 2018 **Document ID:** 213396

[Bias-Free Language](#)

Contents

Introduction

Prerequisites

- Requirements

- Components Used

Background Information

Develop a Python Application with NX-SDK

- Enable NX-SDK

- Create a Python File

- Implement NX-SDK Components

- Create Custom CLI Commands

- pyCmdHandler Class

- Custom CLI Command Syntax Examples

- Single Keyword

- Single Parameter

- Optional Keyword

- Optional Parameter

- Single Keyword and Parameter

- Multiple Keywords and Parameters

- Multiple Keywords and Parameters with Optional Keyword

- Multiple Keywords and Parameters with Optional Parameter

Debug a Python Application with NX-SDK

Deploy a Python Application with NX-SDK

Related Information

Introduction

This document provides a workflow for Python application development with Cisco NX-Software Development Kit (SDK) with the use Cisco NX-OS on the Nexus 3000 and Nexus 9000 platforms.

Prerequisites

Requirements

There are no specific requirements for this document.

Components Used

The information in this document is based on these software and hardware versions:

- This document utilizes NX-SDK v1.0.0 and NX-SDK v1.5.0
- NX-SDK v1.0.0 can be utilized on the Nexus 9000 platform starting from NX-OS Release 7.0(3)I6(1) and the Nexus 3000 platform starting from NX-OS Release 7.0(3)I7(1)
- NX-SDK v1.5.0 can be utilized on both the Nexus 9000 platform and the Nexus 3000 platform starting from NX-OS Release 7.0(3)I7(3)

The information in this document was created from the devices in a specific lab environment. All of the devices used in this document started with a cleared (default) configuration. If your network is live, make sure that you understand the potential impact of any command.

Background Information

Cisco NX-SDK allows for the development of custom applications that can run natively in Cisco NX-OS on Nexus 9000 and Nexus 3000 platforms. NX-SDK offers the ability for customers to create their own CLI commands and outputs, generate customized syslogs in response to specific events, stream tailored telemetry, and much more.

NX-SDK has a C++ API, which is translated into other languages with the use of Simplified Wrapper and Interface Generator (SWIG). This allows for the customer to utilize NX-SDK in any language of their choice. This document demonstrates the implementation of common NX-SDK functions in Python, as well as provides a workflow for customers to develop their own NX-SDK Python applications.

Develop a Python Application with NX-SDK

Enable NX-SDK

In order for any NX-SDK application to run, the NX-SDK feature must first be enabled on the device:

```
switch(config)# feature nxsdk
```

Create a Python File

A Python file can be created and edited with the use of the NX-OS Bash shell. In order for the Bash shell to be used, it must first be enabled on the device:

```
switch(config)# feature bash-shell
```

Enter the Bash shell and use the vi text editor in order to create and edit the Python file:

```
switch(config)# run bash
bash-4.2$ vi /isan/bin/nxsdk-app.py
```

 **Note:** Best practice is to create Python files in the **/isan/bin/** directory. Python files need execution permissions in order to run – do not place Python files in the **/bootflash** directory or any of its subdirectories.

 **Note:** It is not required to create and edit Python files through NX-OS. The developer may create the application using their local environment and transfer the completed files to the device using a file transfer protocol of their choice. However, it might be more efficient for the developer to debug and troubleshoot their script using NX-OS utilities.

Implement NX-SDK Components

It is recommended for developers to begin to create their NX-SDK Python application from the customCliPyApp template on the Cisco DevNet NX-SDK GitHub. These sections of this document refer to necessary components with the use of their names within this template.

There are four major components necessary for an NX-SDK Python application:

1. NX-SDK must be imported into the application via the **import nx_sdk_py** statement.
2. A function (typically named **sdkThread**) that starts the NX-SDK application and modifies various options related to the application.
3. The creation of custom CLI commands as well as the definition of custom CLI command syntax within the **sdkThread** function.
4. A class named **pyCmdHandler** with a method named **postCliCb**, which processes custom CLI commands added by the NX-SDK application.

sdkThread Function

The **sdkThread** function initializes, adds functionality to, and starts the NX-SDK application. The function does not require any parameters to be passed into it. All Python NX-SDK applications require three methods from the **nx_sdk_py** library to be called:

1. **nx_sdk_py.NxSdk.getSdkInst(len(sys.argv), sys.argv)** either returns an SDK instance object, or returns None. If an SDK instance object is returned, then the NX-SDK application has successfully registered with the NX-OS infrastructure. If None is returned, then errors occur at the time of this registration process, and an Error log entry appears in the device's syslog. This method is documented [here](#).

 **Note:** Starting from NX-SDK v1.5.0, a third Boolean parameter can be passed into the **NxSdk.getSdkInst** method, which enables Advanced Exceptions when True and disables Advanced Exceptions when False. This method is documented [here](#) .

1. **sdk.startEventLoop()** method, where **sdk** is the SDK instance object returned by the **nx_sdk_py.NxSdk.getSdkInst()** method. This method starts the NX-SDK application and allows it to interact with the NX-OS infrastructure. This method is documented [here](#) .

2. `nx_sdk_py.NxSdk.__swig_destroy__(sdk)` method, where `sdk` is the SDK instance object returned by the `nx_sdk_py.NxSdk.getSdkInst()` method explained previously. This method placed at the end of the `sdkThread` function allows for the graceful exit of the NX-SDK application.

Some commonly-used methods include:

- `sdk.getTracer()`, where `sdk` is the SDK instance object returned by the `nx_sdk_py.NxSdk.getSdkInst()` method. This method returns an `NxTrace` object which can be used to generate custom syslogs, as well as log events and errors to the application's event history. Custom syslogs will appear in the device's syslog (visible via the **show logging logfile** command) while the events logged to the application's event history are visible via the **show <application-name> nxsdk event-history events** or **show <application-name> nxsdk event-history errors** commands. This method is documented here. The `NxTrace` object returned by this method and its associated methods are documented here. For example, you can view the event history of an application named `Transceiver_DOM.py` through the **show Transceiver_DOM.py nxsdk event-history events** and **show Transceiver_DOM.py nxsdk event-history errors** commands.
- `sdk.getCliParser()`, where `sdk` is the SDK instance object returned by the `nx_sdk_py.NxSdk.getSdkInst()` method. This method returns an `NxCliParser` object, which can be used to execute the CLI commands that already exist via Python as well as create custom CLI commands. This method is documented here. The `NxCliParser` object returned by this method and its associated methods are documented here .
- `cliP.execShowCmd("cmd", return_type)`, where `cliP` is the `NxCliParser` object returned by the `sdk.getCliParser()` method, `cmd` is the show command that you wish to execute encapsulated in quotation marks, and `return_type` is the data format to be output. The data format can be either `R_TEXT`, `R_JSON`, or `R_XML`. This method is documented here .

 **Note:** The `R_JSON` and `R_XML` data formats only work if the command supports output in those formats. In NX-OS, you can verify whether a command supports output in a particular data format by piping the output to the requested data format. If the piped command returns meaningful output, then that data format is supported. For example, if you run **show mac address-table dynamic | json** in NX-OS returns JSON output, then the `R_JSON` data format is supported in NX-SDK as well.

- `cliP.execConfigCmd(cmd_filename)`, where `cliP` is the `NxCliParser` object returned by the `sdk.getCliParser()` method, and `cmd_filename` is the absolute filepath to a file that contains commands separated by lines. This method returns a string that indicates the success of command execution - if "SUCCESS" is in the string, then all commands successfully get executed. Otherwise, the string contains the exception that describes why the commands failed to execute. This method is documented here.

Some optional methods that can be helpful are:

- `sdk.setAppDesc('description string')`, where `sdk` is the SDK instance object returned by the `nx_sdk_py.NxSdk.getSdkInst()` method. This method sets the description of the NX-SDK application. The description is displayed in the NX-OS context-sensitive help menu accessed via a question mark on the CLI. This method is documented here. For example, an application named `Transceiver_DOM.py`, an application description of Returns all interfaces with DOM-capable transceivers inserted appears in the NX-OS context-sensitive help as follows:

```
N9K-C93180LC-EX# show Tra?
track                               Tracking information
Transceiver_DOM.py                 Returns all interfaces with DOM-capable transceivers
inserted
```

- `sdk.getAppname()`, where `sdk` is the SDK instance object returned by the `nx_sdk_py.NxSdk.getSdkInst()` method. This method returns the name of the Python application. This method is documented here.

Create Custom CLI Commands

In a Python application with the use of NX-SDK, custom CLI commands are created and defined within the `sdkThread` function. There are two types of commands: **Show** commands, and **Config** commands.

1. **Show** commands display information about the device, its configuration, or its environment.
2. **Config** commands change the device's configuration, which modifies how the device reacts to the surrounding network.

These two methods allow the creation of show commands and config commands respectively:

- `cliP.newShowCmd("cmd_name", "syntax")`, where `cliP` is the `NxCliParser` object returned by the `sdk.getCliParser()` method, `cmd_name` is a unique name for the command internal to the custom NX-SDK application, and `syntax` describes what keywords and parameters can be used in the command. This method returns an `NxCliCmd` object, which is documented [here](#). This method is documented [here](#).

 **Note:** This command is a subclass of `cliP.newCliCmd("cmd_type", "cmd_name", "syntax")` where `cmd_type` is either `CONF_CMD` or `SHOW_CMD` (depending on the type of command being configured), `cmd_name` is a unique name for the command internal to the custom NX-SDK application, and `syntax` describes what keywords and parameters can be used in the command. Because of this, the API documentation for this command might be more helpful for reference.

- `cliP.newConfigCmd("cmd_name", "syntax")`, where `cliP` is the `NxCliParser` object returned by the `sdk.getCliParser()` method, `cmd_name` is a unique name for the command internal to the custom NX-SDK application, and `syntax` describes what keywords and parameters can be used in the command. This method returns an `NxCliCmd` object, which is documented [here](#). This method is documented [here](#).

 **Note:** This command is a subclass of `cliP.newCliCmd("cmd_type", "cmd_name", "syntax")` where `cmd_type` is either `CONF_CMD` or `SHOW_CMD` (it depends on the type of command that is configured), `cmd_name` is a unique name for the command internal to the custom NX-SDK application, and `syntax` describes what keywords and parameters can be used in the command. Because of this, the API documentation for this command might be more helpful for reference.

Both types of commands have two different components: Parameters and Keywords:

1. **Parameters** are values used to change the results of the command. For example, in the command `show ip route 192.168.1.0`, there is a `route` keyword followed by a parameter that accepts an IP address, which specifies that only routes that include the provided IP address should be shown.

2. **Keywords** change the results of the command through their presence alone. For example, in the command `show mac address-table dynamic`, there is a `dynamic` keyword, which specifies that only dynamically-learned MAC addresses are to be displayed.

Both components are defined in the syntax of an NX-SDK command when it is created. Methods for the `NxCliCmd` object exist to modify the specific implementation of both the components.

- `nx_cmd.updateParam("<parameter>", "help_str", type)`, where `nx_cmd` is the `NxCliCmd` object returned by the `cliP.newShowCmd()` or the `cliP.newConfigCmd()` methods, `<parameter>` is the name of the command parameter that can be modified enclosed by angular brackets (`<>`), `help_str` sets the help-string of the custom command and is displayed in the NX-OS context-sensitive help menu accessed via a question mark on the CLI, and `type` is the type of the parameter. Additional optional parameters for this method are available and documented [here](#). These are valid types for parameters that can be specified in the `type` argument:

- **P_INTEGER** - specifies any integer
- **P_STRING** - specifies any string
- **P_INTERFACE** - specifies any network interface

- **P_IP_ADDR** - specifies any IP address
- **P_MAC_ADDR** - specifies any MAC address
- **P_VRF** - specifies any Virtual Routing and Forwarding (VRF) instance
- **nx_cmd.updateKeyword("keyword", "help_str", is_key)**, where **nx_cmd** is the **NxCliCmd** object returned by the **cliP.newShowCmd()** or the **cliP.newConfigCmd()** methods, **keyword** is the name of the command keyword that you wish to modify, **help_str** sets the help-string of the custom command and is displayed in the NX-OS context-sensitive help menu accessed via a question mark on the CLI, and **is_key** is an optional Boolean value that defaults to False. If **is_key** is True, then unique configuration created by the command with the use of this keyword does not overwrite other unique configuration that is created by the command. If **is_key** is False, then configuration created by the command with the use of this keyword overwrites the other configuration created by the command. This method is documented [here](#) .

In order to view code examples of commonly-used command components, view the Custom CLI Command Examples section of this document.

After custom CLI commands have been created, an object from the **pyCmdHandler** class described later in this document needs to be created and set as the CLI callback handler object for the **NxCliParser** object. This is demonstrated as follows:

```
cmd_handler = pyCmdHandler()
cliP.setCmdHandler(cmd_handler)
```

Then, the **NxCliParser** object needs to be added to the NX-OS CLI parser tree so that custom CLI commands are visible to the user. This is done with the **cliP.addToParseTree()** command, where **cliP** is the **NxCliParser** object returned by the **sdk.getCliParser()** method.

sdkThread Function Example

Here is an example of a typical **sdkThread** function with the use of the functions explained previously. This function (among others within a typical custom NX-SDK Python application) utilizes global variables, which are instantiated on script execution.

```
cliP = ""
sdk = ""
event_hdlr = ""
tmsg = ""

def sdkThread():
    global cliP, sdk, event_hdlr, tmsg

    sdk = nx_sdk_py.NxSdk.getSdkInst(len(sys.argv), sys.argv)
    if not sdk:
        return

    sdk.setAppDesc("Returns all interfaces with DOM-capable transceivers
inserted")

    tmsg = sdk.getTracer()
    tmsg.event("[{}] Started service".format(sdk.getAppname()))

    cliP = sdk.getCliParser()

    nxcmd = cliP.newShowCmd("show_port_bw_util_cmd", "port bw utilization
```

```
[<port>]")
    nxcmd.updateKeyword("port", "Port Information")
    nxcmd.updateKeyword("bw", "Port Bandwidth Information")
    nxcmd.updateKeyword("utilization", "Port BW utilization in (%)")
    nxcmd.updateParam("<port>", "Optional Filter Port Ex) Ethernet1/1",
nx_sdk_py.P_INTERFACE)

    nxcmd1 = cliP.newConfigCmd("port_bw_threshold_cmd", "port bw threshold
<threshold>")
    nxcmd1.updateKeyword("threshold", "Port BW Threshold in (%)")

    int_attr = nx_sdk_py.cli_param_type_integer_attr()
    int_attr.min_val = 1;
    int_attr.max_val = 100;
    nxcmd1.updateParam("<threshold>", "Threshold Limit. Default 50%",
nx_sdk_py.P_INTEGER, int_attr, len(int_attr))

    mycmd = pyCmdHandler()
    cliP.setCmdHandler(mycmd)

    cliP.addToParseTree()

    sdk.startEventLoop()

# If sdk.stopEventLoop() is called or application is removed from VSH...
tmsg.event("Service Quitting...!")

nx_sdk_py.NxSdk.__swig_destroy__(sdk)
```

pyCmdHandler Class

The **pyCmdHandler** class is inherited from the **NxCmdHandler** class within the nx_sdk_py library. The **postCliCb(self, clicmd)** method defined within the **pyCmdHandler** class is called whenever CLI commands that originate from an NX-SDK application. Thus, the **postCliCb(self, clicmd)** method is where you define how the custom CLI commands defined within the sdkThread function behave on the device.

The **postCliCb(self, clicmd)** function returns a Boolean value. If **True** is returned, then it is presumed that the command has been executed successfully. **False** should be returned if the command did not execute successfully for any reason.

The **clicmd** parameter uses the unique name that was defined for the command when it was created in the sdkThread function. For example, if you create a new **show** command with a unique name of **show_xcvr_dom**, then it is recommended to refer to this command by the same name in the **postCliCb(self, clicmd)** function after you have checked to see if the clicmd argument's name contains **show_xcvr_dom**. It is demonstrated here:

```
def sdkThread():
    <snip>
    sh_xcvr_dom = cliP.newShowCmd("show_xcvr_dom", "dom")
    sh_xcvr_dom.updateKeyword("dom", "Show all interfaces with transceivers
```



```
that are DOM-capable")
</snip>
```

```
class pyCmdHandler(nx_sdk_py.NxCmdHandler):
    def postCliCb(self, clicmd):
        if "show_xcvr_dom" in clicmd.getCmdName():
            get_dom_capable_interfaces()
```

If a command that utilizes parameters is created, then you will most likely need to use those parameters at some point in the **postCliCb(self, clicmd)** function. This can be done with the **clicmd.getParamValue("<parameter>")** method, where **<parameter>** is the name of the command parameter that you wish to get the value of enclosed by angular brackets (<>). This method is documented [here](#). However, the value returned by this function needs to be converted to the type that you need. This can be done with these methods:

- **nx_sdk_py.void_to_int** converts a value to an integer type.
- **nx_sdk_py.void_to_string** converts a value to a string type.

The **postCliCb(self, clicmd)** function (or any subsequent functions) will also typically be where show command output is printed to the console. This is done with the **clicmd.printConsole()** method.

 **Note:** If the application encounters an error, unhandled exception, or otherwise suddenly exits, then output from the **clicmd.printConsole()** function will not be displayed at all. For this reason, best practice when you debug your Python application is to either log debug messages to the syslog with the use of an NxTrace object returned by the **sdk.getTracer()** method, or use print statements and execute the application via the Bash shell's **/isan/bin/python** binary.

pyCmdHandler Class Example

The following code serves as an example for the **pyCmdHandler** class described above. This code is taken from the `ip_move.py` file in the ip-movement NX-SDK application available [here](#). The purpose of this application is to track the movement of a user-defined IP address across interfaces of a Nexus device. To do this, the code finds the MAC address of the IP address input through the **<ip>** parameter within the device's ARP cache, then verifies which VLAN that MAC address resides in using the device's MAC address table. Using this MAC and VLAN, the **show system internal l2fm l2dbg macdb address <mac> vlan <vlan>** command displays a list of SNMP interface indexes that this combination has recently been associated with. The code then uses the **show interface snmp-ifindex** command to translate recent SNMP interface indexes into human-legible interface names.

```
class pyCmdHandler(nx_sdk_py.NxCmdHandler):
    def postCliCb(self, clicmd):
        global cli_parser

        if "show_ip_movement" in clicmd.getCmdName():
            target_ip = nx_sdk_py.void_to_string(clicmd.getParamValue("<ip>"))

            target_mac = get_mac_from_arp(cli_parser, clicmd, target_ip)
            mac_vlan = ""
            if target_mac:
                mac_vlan = get_vlan_from_cam(cli_parser, clicmd, target_mac)
                if mac_vlan:
                    find_mac_movement(cli_parser, clicmd, target_mac, mac_vlan)
            else:
```



```

        print("No entires in MAC address table")
        clicmd.printConsole("No entries in MAC address table for
{}".format(target_mac))
    else:
        clicmd.printConsole("No entries in ARP table for
{}".format(target_ip))
    return True

def get_mac_from_arp(cli_parser, clicmd, target_ip):
    exec_cmd = "show ip arp {}".format(target_ip)
    arp_cmd = cli_parser.execShowCmd(exec_cmd, nx_sdk_py.R_JSON)
    if arp_cmd:
        try:
            arp_json = json.loads(arp_cmd)
        except ValueError as exc:
            return None
        count = int(arp_json["TABLE_vrf"]["ROW_vrf"]["cnt-total"])
        if count:
            intf = arp_json["TABLE_vrf"]["ROW_vrf"]["TABLE_adj"]["ROW_adj"]
            if intf.get("ip-addr-out") == target_ip:
                target_mac = intf["mac"]
                clicmd.printConsole("{} is currently present in ARP table, MAC
address {}\n".format(target_ip, target_mac))
                return target_mac
            else:
                return None
        else:
            return None
    else:
        return None

def get_vlan_from_cam(cli_parser, clicmd, target_mac):
    exec_cmd = "show mac address-table address {}".format(target_mac)
    mac_cmd = cli_parser.execShowCmd(exec_cmd, nx_sdk_py.R_JSON)
    if mac_cmd:
        try:
            cam_json = json.loads(mac_cmd)
        except ValueError as exc:
            return None
        mac_entry = cam_json["TABLE_mac_address"]["ROW_mac_address"]
        if mac_entry:
            if mac_entry["disp_mac_addr"] == target_mac:
                egress_intf = mac_entry["disp_port"]
                mac_vlan = mac_entry["disp_vlan"]
                clicmd.printConsole("{} is currently present in MAC address
table on interface {}, VLAN {}\n".format(target_mac, egress_intf, mac_vlan))
                return mac_vlan
            else:

```

```

        return None
    else:
        return None
else:
    return None

def find_mac_movement(cli_parser, clicmd, target_mac, mac_vlan):
    exec_cmd = "show system internal l2fm l2dbg macdb address {} vlan
{}".format(target_mac, mac_vlan)
    l2fm_cmd = cli_parser.execShowCmd(exec_cmd)
    if l2fm_cmd:
        event_re = re.compile(r"^s+(\w{3}) (\w{3}) (\d+) (\d{2}):(\d{2}):
(\d{2}) (\d{4}) (0x\S{8}) (\d+)\s+(\S+) (\d+)\s+(\d+)\s+(\d+)")
        unique_interfaces = []
        l2fm_events = l2fm_cmd.splitlines()
        for line in l2fm_events:
            res = re.search(event_re, line)
            if res:
                day_name = res.group(1)
                month = res.group(2)
                day = res.group(3)
                hour = res.group(4)
                minute = res.group(5)
                second = res.group(6)
                year = res.group(7)
                if_index = res.group(8)
                db = res.group(9)
                event = res.group(10)
                src=res.group(11)
                slot = res.group(12)
                fe = res.group(13)
                if "MAC_NOTIF_AM_MOVE" in event:
                    timestamp = "{} {} {} {}: {}: {} {}".format(day_name, month,
day, hour, minute, second, year)
                    intf_dict = {"if_index": if_index, "timestamp": timestamp}
                    unique_interfaces.append(intf_dict)
            if not unique_interfaces:
                clicmd.printConsole("No entries for {} in L2FM
L2DBG\n".format(target_mac))
            if len(unique_interfaces) == 1:
                clicmd.printConsole("{} has not been moving between
interfaces\n".format(target_mac))
            if len(unique_interfaces) > 1:
                clicmd.printConsole("{} has been moving between the following
interfaces, from most recent to least recent:\n".format(target_mac))
                unique_interfaces = get_snmp_intf_index(unique_interfaces)
                clicmd.printConsole("\t{} - {} (Current
interface)\n".format(unique_interfaces[-1]["timestamp"], unique_interfaces[-1]

```

```

["intf_name"])))
        for intf in unique_interfaces[-2::-1]:
            clicmd.printConsole("\t{} - {}\n".format(intf["timestamp"],
intf["intf_name"])))

def get_snmp_intf_index(if_index_dict_list):
    global cli_parser

    snmp_ifindex = cli_parser.execShowCmd("show interface snmp-ifindex",
nx_sdk_py.R_JSON)
    snmp_ifindex_json = json.loads(snmp_ifindex)
    snmp_ifindex_list = snmp_ifindex_json["TABLE_interface"]["ROW_interface"]
    for index_dict in if_index_dict_list:
        index = index_dict["if_index"]
        for ifindex_json in snmp_ifindex_list:
            if index == ifindex_json["snmp-ifindex"]:
                index_dict["intf_name"] = ifindex_json["interface"]
    return if_index_dict_list

```

Custom CLI Command Syntax Examples

This section showcases some examples of the syntax parameter used when you create custom CLI commands with the **cliP.newShowCmd()** or the **cliP.newConfigCmd()** methods, where **cliP** is the NxCliParser object returned by the **sdk.getCliParser()** method.

 **Note:** Support for syntax with opening and closing parentheses "(" and ")") is introduced in NX-SDK v1.5.0, included in NX-OS Release 7.0(3)I7(3). It is assumed that the user utilizes NX-SDK v1.5.0 when they follow any of these given examples that include syntax utilizing opening and closing parentheses.

Single Keyword

This show command takes a single keyword mac and adds a helper string of Shows all misprogrammed MAC addresses on this device to the keyword.

```

nx_cmd = cliP.newShowCmd("show_misprogrammed", "mac")
nx_cmd.updateKeyword("mac", "Shows all misprogrammed MAC addresses on this
device")

```

Single Parameter

This show command takes a single parameter **<mac>**. The enclosing angle brackets around the word mac signify that this is a parameter. A helper string of MAC address to check for misprogramming is added to the parameter. The **nx_sdk_py.P_MAC_ADDR** parameter in the **nx_cmd.updateParam()** method is used to define the type of the parameter as a MAC address, which prevents end user input of another type, such as a string, integer, or IP address.

```

nx_cmd = cliP.newShowCmd("show_misprogrammed_mac", "<mac>")
nx_cmd.updateParam("<mac>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)

```

Optional Keyword

This show command may optionally take a single keyword **[mac]**. The enclosing brackets around the word mac signify that this keyword is optional. A helper string of Shows all misprogrammed MAC addresses on this device is added to the keyword.

```
nx_cmd = cliP.newShowCmd( "show_misprogrammed_mac" , "[mac]" )
nx_cmd.updateKeyword( "mac" , "Shows all misprogrammed MAC addresses on
this device" )
```

Optional Parameter

This show command may optionally take a single parameter **[<mac>]**. The enclosing brackets around the word < mac > signify that this parameter is optional. The enclosing angle brackets around the word mac signify that this is a parameter. A helper string of MAC address to check for misprogramming is added to the parameter. The **nx_sdk_py.P_MAC_ADDR** parameter in the **nx_cmd.updateParam()** method is used to define the type of the parameter as a MAC address, which prevents end user input of another type, such as a string, integer, or IP address.

```
nx_cmd = cliP.newShowCmd("show_misprogrammed_mac", "[<mac>]")
nx_cmd.updateParam("<mac>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)
```

Single Keyword and Parameter

This show command takes a single keyword mac immediately followed by the parameter **<mac-address>**. The enclosing angle brackets around the word mac-address signify that this is a parameter. A helper string of Check MAC address for misprogramming is added to the keyword. A helper string of MAC address to check for misprogramming is added to the parameter. The **nx_sdk_py.P_MAC_ADDR** parameter in the **nx_cmd.updateParam()** method is used in order to define the type of the parameter as a MAC address, which prevents end user input of another type, such as a string, integer, or IP address.

```
nx_cmd = cliP.newShowCmd("show_misprogrammed", "mac <mac-address>")
nx_cmd.updateKeyword("mac", "Check MAC address for misprogramming")
nx_cmd.updateParam("<mac-address>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)
```

Multiple Keywords and Parameters

This show command can take one of two keywords, both of which have two different parameters following them. The first keyword mac has a parameter of **<mac-address>**, and the second keyword ip has a parameter of **<ip-address>**. The enclosing angle brackets around the words mac-address and ip-address signify that they are parameters. A helper string of Check MAC address for misprogramming is added to the mac keyword. A helper string of MAC address to check for misprogramming is added to the **<mac-address>** parameter. The **nx_sdk_py.P_MAC_ADDR** parameter in the **nx_cmd.updateParam()** method is used to define the type of the **<mac-address>** parameter as a MAC address, which prevents the end user input of another type, such as a string, integer, or IP address. A helper string of Check IP address for misprogramming is added to the ip keyword. A helper string of IP address to check for misprogramming is added to the **<ip-address>** parameter. The **nx_sdk_py.P_IP_ADDR** parameter in the **nx_cmd.updateParam()** method is used to define the type of the **<ip-address>** parameter as an IP address, which prevents the end user input of another type, such as a string, integer, or IP address.

```

nx_cmd = cliP.newShowCmd("show_misprogrammed", "(mac <mac-address> | ip <ip-
address>)")
nx_cmd.updateKeyword("mac", "Check MAC address for misprogramming")
nx_cmd.updateParam("<mac-address>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)
nx_cmd.updateKeyword("ip", "Check IP address for misprogramming")
nx_cmd.updateParam("<ip-address>", "IP address to check for misprogramming",
nx_sdk_py.P_IP_ADDR)

```

Multiple Keywords and Parameters with Optional Keyword

This show command can take one of two keywords, both of which have two different parameters following them. The first keyword `mac` has a parameter of **<mac-address>**, and the second keyword `ip` has a parameter of **<ip-address>**. The enclosing angle brackets around the words `mac-address` and `ip-address` signify that they are parameters. A helper string of Check MAC address for misprogramming is added to the `mac` keyword. A helper string of MAC address to check for misprogramming is added to the **<mac-address>** parameter. The `nx_sdk_py.P_MAC_ADDR` parameter in the `nx_cmd.updateParam()` method is used to define the type of the **<mac-address>** parameter as a MAC address, which prevents the end user input of another type, such as a string, integer, or IP address. A helper string of Check IP address for misprogramming is added to the `ip` keyword. A helper string of IP address to check for misprogramming is added to the **<ip-address>** parameter. The `nx_sdk_py.P_IP_ADDR` parameter in the `nx_cmd.updateParam()` method is used to define the type of the **<ip-address>** parameter as an IP address, which prevents the end user input of another type, such as a string, integer, or IP address. This show command might optionally take a keyword `[clear]`. A helper string Clears addresses detected to be misprogrammed is added to this optional keyword.

```

nx_cmd = cliP.newShowCmd("show_misprogrammed", "(mac <mac-address> | ip <ip-
address>) [clear]")
nx_cmd.updateKeyword("mac", "Check MAC address for misprogramming")
nx_cmd.updateParam("<mac-address>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)
nx_cmd.updateKeyword("ip", "Check IP address for misprogramming")
nx_cmd.updateParam("<ip-address>", "IP address to check for misprogramming",
nx_sdk_py.P_IP_ADDR)
nx_cmd.updateKeyword("clear", "Clears addresses detected to be misprogrammed")

```

Multiple Keywords and Parameters with Optional Parameter

This show command can take one of two keywords, both of which have two different parameters following them. The first keyword `mac` has a parameter of **<mac-address>**, and the second keyword `ip` has a parameter of **<ip-address>**. The enclosing angle brackets around the words `mac-address` and `ip-address` signify that they are parameters. A helper string of Check MAC address for misprogramming is added to the `mac` keyword. A helper string of MAC address to check for misprogramming is added to the **<mac-address>** parameter. The `nx_sdk_py.P_MAC_ADDR` parameter in the `nx_cmd.updateParam()` method is used to define the type of the **<mac-address>** parameter as a MAC address, which prevents the end user input of another type, such as a string, integer, or IP address. A helper string of Check IP address for misprogramming is added to the `ip` keyword. A helper string of IP address to check for misprogramming is added to the **<ip-address>** parameter. The `nx_sdk_py.P_IP_ADDR` parameter in the `nx_cmd.updateParam()` method is used to define the type of the **<ip-address>** parameter as an IP address, which prevents the end user input of another type, such as a string, integer, or IP address. This show command might optionally take a parameter **[<module>]**. A helper string Only clear addresses on specified module is added to this optional parameter.

```

nx_cmd = cliP.newShowCmd("show_misprogrammed", "(mac <mac-address> | ip <ip-
address>) [<module>]")
nx_cmd.updateKeyword("mac", "Check MAC address for misprogramming")
nx_cmd.updateParam("<mac-address>", "MAC address to check for misprogramming",
nx_sdk_py.P_MAC_ADDR)
nx_cmd.updateKeyword("ip", "Check IP address for misprogramming")
nx_cmd.updateParam("<ip-address>", "IP address to check for misprogramming",
nx_sdk_py.P_IP_ADDR)
nx_cmd.updateParam("<module>", "Clears addresses detected to be
misprogrammed", nx_sdk_py.P_INTEGER)

```

Debug a Python Application with NX-SDK

Once an NX-SDK Python application has been created, it will often need to be debugged. NX-SDK informs you in case there are any syntax errors in your code, but because the Python NX-SDK library utilizes SWIG to translate C++ libraries to Python libraries, any exceptions encountered at the time of the code execution result in an application core dump similar to this:

```

terminate called after throwing an instance of 'Swig::DirectorMethodException'
  what(): SWIG director method error. Error detected when calling
'NxCmdHandler.postCliCb'
Aborted (core dumped)

```

Due to the ambiguous nature of this error message, best practice to debug Python applications is to log debug messages to the syslog with the use of an NxTrace object returned by the **sdk.getTracer()** method. This is demonstrated as follows:

```

#!/isan/bin/python

tracer = 0

def evt_thread():
    <snip>
    tracer = sdk.getTracer()
    tracer.event("[NXSDK-APP][INFO] Started service")
<snip>
class pyCmdHandler(nx_sdk_py.NxCmdHandler):
    def postCliCb(self, clicmd):
        global tracer
        tracer.event("[NXSDK-APP][DEBUG] Received command: {}".format(clicmd))
        if "show_test_command" in clicmd.getCmdName():
            tracer.event("[NXSDK-APP][DEBUG] `show_test_command` recognized")

```

If logging debug messages to the syslog is not an option, an alternative method is to use print statements and execute the application via the Bash shell's **/isan/bin/python** binary. However, the output from these print statements will only be visible when executed in this manner - running the application through the VSH shell does not produce any output. An example of utilizing print statements is shown here:

```

#!/isan/bin/python

tracer = 0

```

```
def evt_thread():
    <snip>
    print("[NXSDK-APP][INFO] Started service")
<snip>
class pyCmdHandler(nx_sdk_py.NxCmdHandler):
    def postCliCb(self, clicmd):
        print("[NXSDK-APP][DEBUG] Received command: {}".format(clicmd))
        if "show_test_command" in clicmd.getCmdName():
            print("[NXSDK-APP][DEBUG] `show_test_command` recognized")
```

Deploy a Python Application with NX-SDK

Once a Python application has been fully tested in the Bash shell and is ready for deployment, the application should be installed into production through VSH. This allows the application to persist when the device reloads or when system switchover occurs in a dual-supervisor scenario. In order to deploy an application through VSH, you need to create an RPM package with the use of an NX-SDK and ENXOS SDK build environment. Cisco DevNet provides a Docker image that allows for easy RPM package creation.

 **Note:** For assistance in order to install Docker on your specific operating system, refer to Docker's installation documentation.

On a Docker-capable host, pull the image version of your choice with the **docker pull dockercisco/nxsdk:<tag>** command, where **<tag>** is the tag of the image version of your choice. You can view the available image versions and their corresponding tags here. This is demonstrated with the **v1** tag here:

```
docker pull dockercisco/nxsdk:v1
```

Start a container named **nxsdk** from this image and attach to it. If the tag of your choice is different, substitute **v1** for your tag:

```
docker run -it --name nxsdk dockercisco/nxsdk:v1 /bin/bash
```

Update to the latest version of NX-SDK and navigate to the **NX-SDK** directory, then pull the latest files from git:

```
cd /NX-SDK/
git pull
```

If you need to use an older version of NX-SDK, you can clone the NX-SDK branch with the use of the respective version tag with the **git clone -b v<version> https://github.com/CiscoDevNet/NX-SDK.git** command, where **<version>** is the version of NX-SDK you need. This is demonstrated here with NX-SDK v1.0.0:

```
cd /
rm -rf /NX-SDK
git clone -b v1.0.0 https://github.com/CiscoDevNet/NX-SDK.git
```

Next, transfer your Python application to the Docker container. There are a few different ways to do this.

- Exit the Docker container (which stops the container and requires you to start it once more), transfer the Python application to the Docker host, then use the **docker cp** command in order to copy the application from the host to the container. This is demonstrated here, under the assumption that the Python application has been transferred to the Docker host at **/app/python_app.py**.


```

root@2dcbe841742a:~# exit
[root@localhost ~]# docker cp /app/python_app.py nxsdk:/root/
[root@localhost ~]# docker start nxsdk
nxsdk
[root@localhost ~]# docker attach nxsdk
root@2dcbe841742a:/# ls /root/
python_app.py

```

- Copy the contents of the Python application into your system clipboard, then paste the contents into a file created in the Docker container with the use of vim.

Next, use the **rpm_gen.py** script located in **/NX-SDK/scripts/** in order to create an RPM package from the Python application. This script has one required argument, and two required switches:

- The filename of the Python application. For example, a Python application in a file named **python_app.py** would result in an argument of **python_app.py**. This filename will later be used as the application name for NX-SDK, and will also be used by NX-OS in order to refer to commands created by this application.

 **Note:** The filename does not need to contain any file extensions, such as **.py**. In this example, if the filename was **python_app** instead of **python_app.py**, the RPM package would be generated without an issue.

- The **-s** switch takes an argument for the absolute filepath that leads to where the aforementioned filename is located. For example, if **python_app.py** is located in **/root/**, then the correct argument would be **-s /root/**.
- The **-u** switch indicates that the source filename is the same as the executable filename.

Usage of the **rpm_gen.py** script is demonstrated here.

```

root@7bfd1714dd2f:~# python /NX-SDK/scripts/rpm_gen.py test_python_app -s
/root/ -u
#####
Generating rpm package...
<snip>
RPM package has been built
#####

SPEC file: /NX-SDK/rpm/SPECS/test_python_app.spec
RPM file  : /NX-SDK/rpm/RPMS/test_python_app-1.0-1.0.0.x86_64.rpm

```

The filepath to the RPM package is indicated in the final line of the **rpm_gen.py** script output. This file must be copied off of the Docker container onto the host so that it can be transferred to the Nexus device that you wish to run the application on. After you exit the Docker container, it can be done easily with the **docker cp <container>:<container_filepath> <host_filepath>** command, where **<container>** is the name of the NX-SDK Docker container (in this case, **nxsdk**), **<container_filepath>** is the full filepath of the RPM package inside the container (in this case, **/NX-SDK/rpm/RPMS/test_python_app-1.0-1.0.0.x86_64.rpm**), and **<host_filepath>** is the full filepath on our Docker host where the RPM package is to be transferred to (in this case, **/root/**). This command is demonstrated here:

```

root@7bfd1714dd2f:/# exit
[root@localhost ~]# docker cp nxsdk:/NX-SDK/rpm/RPMS/test_python_app-1.0-
1.0.0.x86_64.rpm /root/

```

```
[root@localhost ~]# ls /root/
anaconda-ks.cfg          test_python_app-1.0-1.0.0.x86_64.rpm
```

Transfer this RPM package to the Nexus device with the use of your preferred method of file transfer. Once the RPM package is on the device, it must be installed and activated similarly to a SMU. This is demonstrated as follows, under the assumption that the RPM package was transferred to the bootflash of the device.

```
N9K-C93180LC-EX# install add bootflash:test_python_app-1.0-1.0.0.x86_64.rpm
[#####] 100%
Install operation 27 completed successfully at Tue May  8 06:40:13 2018
N9K-C93180LC-EX# install activate test_python_app-1.0-1.0.0.x86_64
[#####] 100%
Install operation 28 completed successfully at Tue May  8 06:40:20 2018
```

 **Note:** When you install the RPM package with the **install add** command, include the storage device and exact filename of the package. When you activate the RPM package after installation, do not include the storage device and filename - use the name of the package itself. You can verify the package name with the **show install inactive** command.

Once the RPM package is activated, you can start the application with NX-SDK with the **nxsdk service <application-name>** configuration command, where **<application-name>** is the name of the Python filename (and, subsequently, the application) that was defined when rpm_gen.py script was used earlier. This is demonstrated as follows:

```
N9K-C93180LC-EX# conf
Enter configuration commands, one per line. End with CNTL/Z.
N9K-C93180LC-EX(config)# nxsdk service-name test_python_app
% This could take some time. "show nxsdk internal service" to check if your App
is Started & Running
```

You can verify that the application is up and has started to run with the **show nxsdk internal service** command:

```
N9K-C93180LC-EX# show nxsdk internal service

NXSDK Started/Temp unavailabe/Max services : 1/0/32
NXSDK Default App Path                     : /isan/bin/nxsdk
NXSDK Supported Versions                   : 1.0
```

Service-name Package	Base App	Started(PID)	Version	RPM
test_python_app	nxsdk_app4	VSH(23195)	1.0	
test_python_app-1.0-1.0.0.x86_64				

You can also verify that custom CLI commands created by this application are accessible in NX-OS:

```
N9K-C93180LC-EX# show test?
test_python_app    Nexus Sdk Application
```

Related Information

- [NX-SDK GitHub](#)
- [Cisco Nexus 9000 Series NX-OS Programmability Guide, Release 7.x](#)
- [Cisco Nexus 3000 Series NX-OS Programmability Guide, Release 7.x](#)
- [Cisco Nexus 3500 Series NX-OS Programmability Guide, Release 7.x](#)
- [Network Programmability and Automation with Cisco Nexus 9000 Series Switches White Paper](#)
- [Programmability and Automation with Cisco Open NX-OS \(PDF\)](#)
- [Technical Support & Documentation - Cisco Systems](#)

Quick Links

[About Cisco](#)

[Contact Us](#)

[Careers](#)

[Connect with a partner](#)

Resources and Legal

[Feedback](#)

[Help](#)

[Terms & Conditions](#)

[Privacy](#)

[Cookies / Do not sell or share my personal data](#)

[Accessibility](#)

[Trademarks](#)

[Supply Chain Transparency](#)

[Newsroom](#)

[Sitemap](#)

